

Секционирование таблиц

Под **секционированием (partitioning)** понимают разделение одной логической таблицы на отдельные физические части (секции). Секции – это изолированные таблицы, развернутые как дочерние от главной (секционируемой) таблицы. При этом для пользователей приложения таблица выглядит единой. Разделение проводится по строкам таблицы, на основании значений одного или нескольких полей таблицы, которые называют *ключами секционирования*.

Данный механизм используется для оптимизации работы с большими таблицами, число записей которых исчисляется десятками тысяч, потому что в этом случае дает определенные преимущества в работе с данными, а именно:

- Операция удаления неактуальных данных становится максимально быстрой, и не требующей последующего выполнения [VACUUM](#).
- Можно перенести секцию в другое табличное пространство; таким образом, редко используемые данные (архивные) можно переносить на более дешевые (медленные) носители информации.
- Индексы изолированы от операций в других секциях, что также дает прирост производительности:
 - нет блокировок при переиндексации;
 - нет лишних изменений;
 - ширина и глубина индекса меньше, что ускоряет чтение индекса;
 - меньше задержек из-за дробления дерева индекса.
- Секция заведомо содержит данные, удовлетворяющие конкретному условию, что обеспечивает быструю проверку условия на уровне СУБД.
- В секции меньше данных, поэтому кэшируются только наиболее часто запрашиваемые данные. СУБД при указанных условиях выборки сама понимает, из какой секции выбрать данные.

Настройка секционирования

При работе с таблицами возможно как настроить секционирование, так и отказаться от него.

1. Настройка секционирования. Опция настройки секционирования таблицы позволяет задать названия секций и условия, по которому в неё будут отобраны соответствующие записи. В процессе конвертации БД:

- создаются дочерние таблицы — секции;
- для каждой секции создается соответствующее ограничение;
- на секции с основной таблицы "перемещаются" индексы;
- триггеры "перед" остаются на главной таблице, триггеры "после" "перемещаются" на секции; на главной таблице создается триггер `z$ins_part`, вызывающий автогенерируемую триггерную функцию "ins_Table" (функция реализует распределение записей по секциям).

2. Отказ от секционирования. Для настройки достаточно удалить все описанные ранее секции. В процессе конвертации БД:

- удаляются секции с ограничениями;
- на основную таблицу с секций "перемещаются" индексы;
- триггеры "перед" остаются без изменений, триггеры "после" с секций "перемещаются" на основную таблицу;
- из основной таблицы удаляется триггер `z$ins_part`.



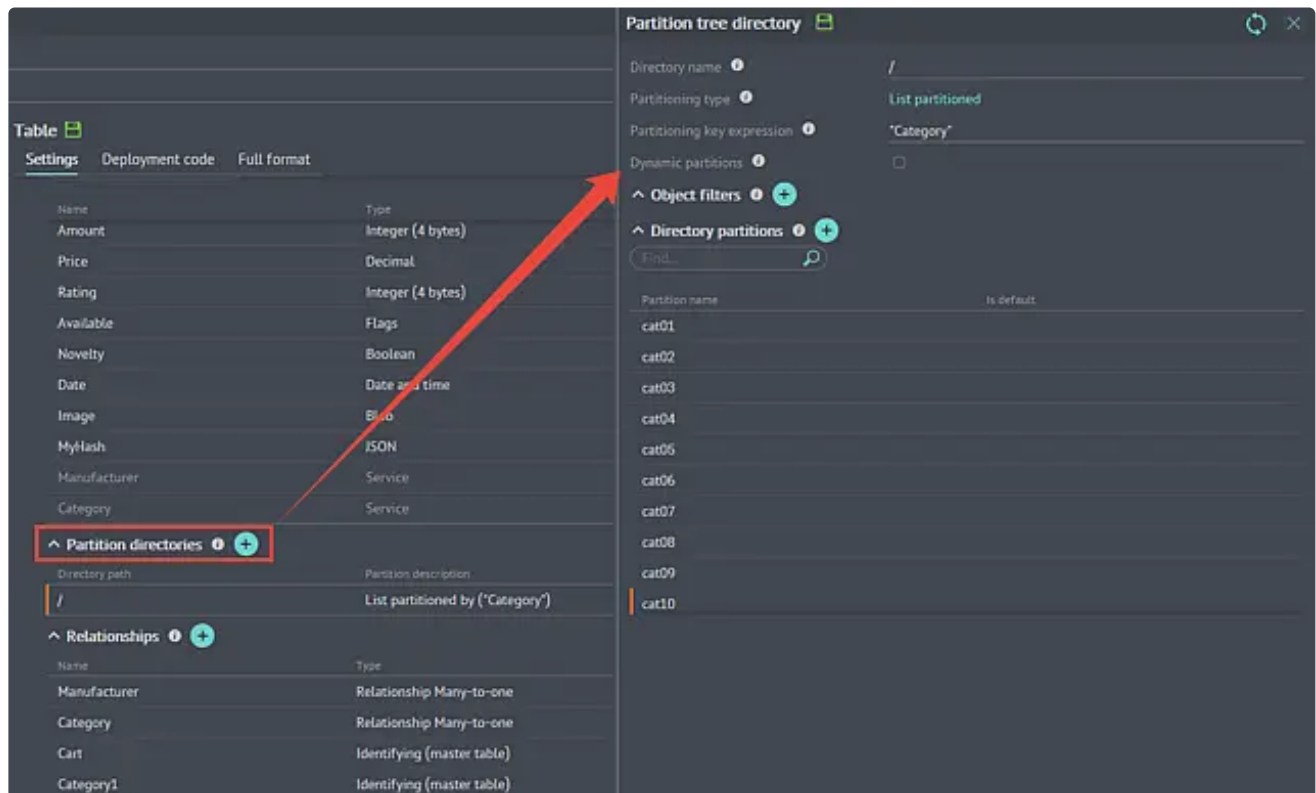
Переход к секционированию, отказ от него, перераспределение секций не подразумевает автоматическое перераспределение уже имеющихся данных в таблице. Внутренних средств PostgreSQL, автоматически переносящих данные, нет, по причине возможного нарушения целостности данных и возможных больших затрат времени.

Задача по управлению существующими записями при изменении структуры решается пользователем с помощью расширений пользовательских конвертеров либо сторонними скриптами перед/после изменения структуры секционируемой таблицы.

Пример настройки

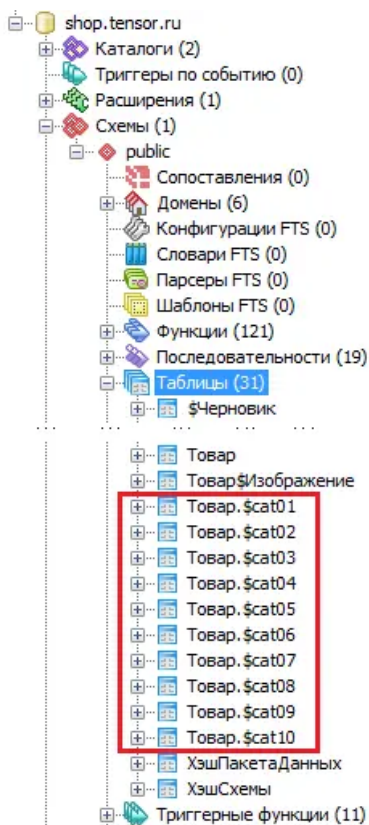
Рассмотрим настройку механизма секционирования на примере веб-приложения для интернет-магазина "Тензор", разработка которого описана в наших [уроках](#).

Предположим, что таблица "Товар" будет содержать большое количество записей, и для более эффективной обработки было принято решение секционировать таблицу. Секционирование проведем по полю "Категория":



Итак, создано десять секций — по количеству категорий.

После конвертации базы данных, в pgAdmin можно посмотреть развернутые секции (дочерние таблицы) для таблицы "Товары" с индексами и ограничением по полю "Категория":



```

CREATE TABLE "Товар.$cat02"
(
    -- Унаследована from table "Товар": "@Товар" integer NOT NULL DEFAULT nextval
    -- Унаследована from table "Товар": "Наименование" text DEFAULT '':text,
    -- Унаследована from table "Товар": "Описание" text DEFAULT '':text,
    -- Унаследована from table "Товар": "Изображение" integer,
    -- Унаследована from table "Товар": "Тип_поставки" smallint,
    -- Унаследована from table "Товар": "Количество" integer,
    -- Унаследована from table "Товар": "Цена" numeric(32,2),
    -- Унаследована from table "Товар": "Рейтинг" integer,
    -- Унаследована from table "Товар": "Наличие_в_магазинах" boolean[] DEFAULT ' ',
    -- Унаследована from table "Товар": "Новинка" boolean,
    -- Унаследована from table "Товар": "Дата_выпуска" timestamp with time zone,
    -- Унаследована from table "Товар": "Производитель" integer,
    -- Унаследована from table "Товар": "Категория" integer,
    CONSTRAINT cat02_check CHECK ("Категория" = 4)
)
INHERITS ("Товар")
WITH (
    OIDS=FALSE
);
ALTER TABLE "Товар.$cat02"
OWNER TO postgres;
COMMENT ON TABLE "Товар.$cat02"
IS 'sbis3_object';

-- Index: "iТовар.$cat02-Категория"
-- DROP INDEX "iТовар.$cat02-Категория";

CREATE INDEX "iТовар.$cat02-Категория"
ON "Товар.$cat02"
USING btree
("Категория");
COMMENT ON INDEX "iТовар.$cat02-Категория"
IS 'sbis3_object';

```

На основе списка секций сгенерирован триггер "z\$ins_part" с триггерной функцией ins_Товар, помещающий записи, удовлетворяющие условиям ограничений секций, в нужную секцию:

```

-- Function: "ins_Товар"()

-- DROP FUNCTION "ins_Товар"();

CREATE OR REPLACE FUNCTION "ins_Товар"()
RETURNS trigger AS
$BODY$
BEGIN
IF ( NEW."Категория" = 2 ) THEN
INSERT INTO "Товар.$cat01" VALUES (NEW.*);
ELSIF ( NEW."Категория" = 4 ) THEN
INSERT INTO "Товар.$cat02" VALUES (NEW.*);
ELSIF ( NEW."Категория" = 5 ) THEN
INSERT INTO "Товар.$cat03" VALUES (NEW.*);
ELSIF ( NEW."Категория" = 6 ) THEN
INSERT INTO "Товар.$cat04" VALUES (NEW.*);
ELSIF ( NEW."Категория" = 7 ) THEN
INSERT INTO "Товар.$cat05" VALUES (NEW.*);
ELSIF ( NEW."Категория" = 8 ) THEN
INSERT INTO "Товар.$cat06" VALUES (NEW.*);
ELSIF ( NEW."Категория" = 9 ) THEN
INSERT INTO "Товар.$cat07" VALUES (NEW.*);
ELSIF ( NEW."Категория" = 10 ) THEN
INSERT INTO "Товар.$cat08" VALUES (NEW.*);
ELSIF ( NEW."Категория" = 11 ) THEN
INSERT INTO "Товар.$cat09" VALUES (NEW.*);
ELSIF ( NEW."Категория" = 12 ) THEN
INSERT INTO "Товар.$cat10" VALUES (NEW.*);
ELSE
RAISE EXCEPTION 'При вставке записи возникло исключение: нет удовлетворяющих критериев для распределения записи.
Переопределите секции таблицы Товар!';
END IF;
RETURN NULL;
END
$BODY$
LANGUAGE plpgsql VOLATILE
COST 100;
ALTER FUNCTION "ins_Товар"()
OWNER TO postgres;
COMMENT ON FUNCTION "ins_Товар"() IS 'sbis3_object';

```

Особенности реализации

Ряд моментов, на которые нужно обратить внимание при использовании секционирования:

- за своевременной организацией новых секций нужно следить; нет возможностей по их автоматическому созданию;
- индексы на дочерние таблицы можно создавать без ограничений, однако сделать один общий индекс для всех секций невозможно;
- нельзя сделать ограничение, первичный и/или уникальный ключ на секционированную таблицу; логической уникальности можно добиться, добавив к первичному ключу каждой секции поле с ключом секционирования;
- для секционирования по диапазонам (дат или чисел) нужно следить за тем, чтобы ограничения CHECK на дочерних таблицах были взаимоисключающими.